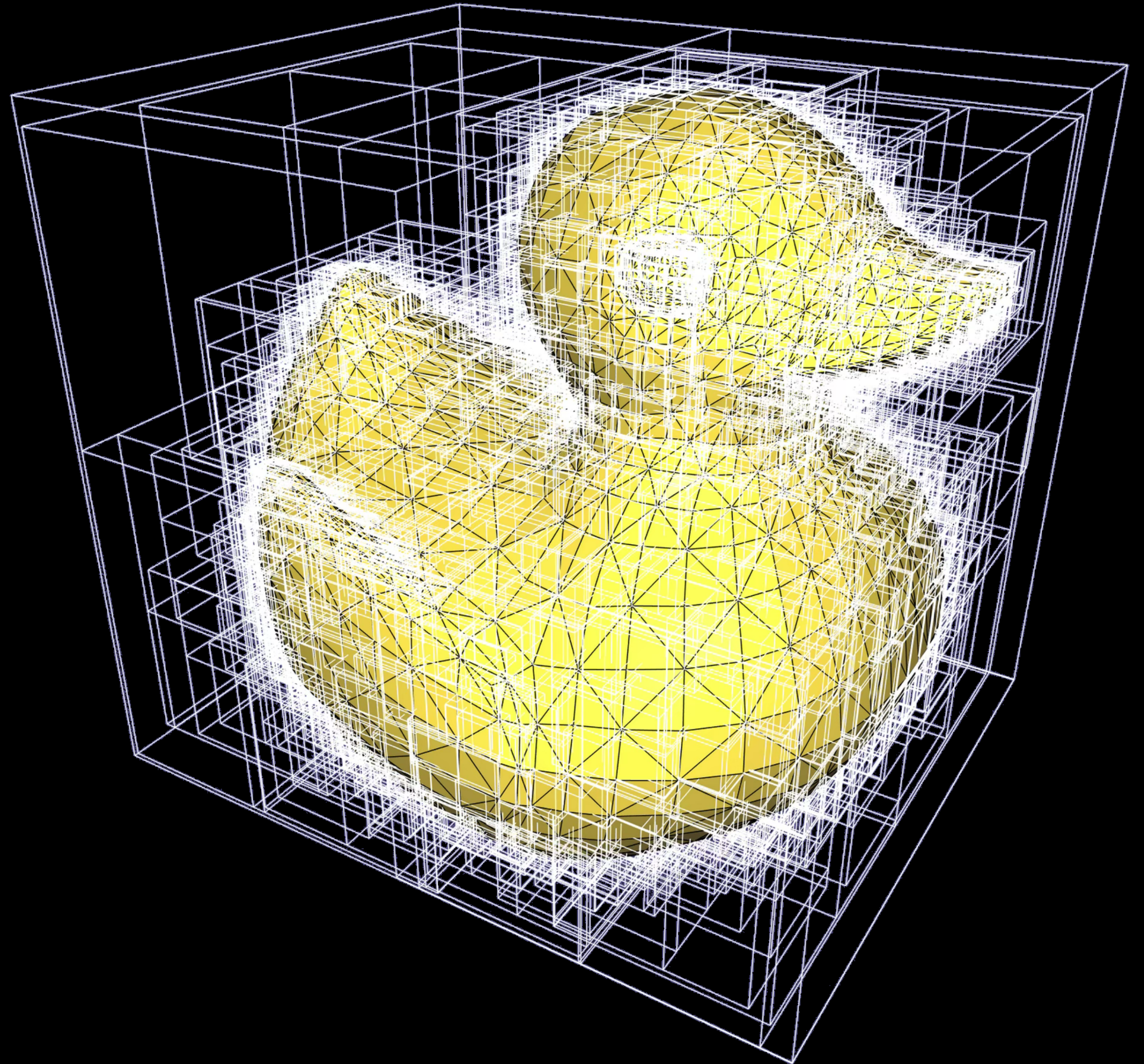


Assignment #4

Bounding Volume Hierarchy



Assignment #3 (Raytracing)

*due **tomorrow**, **Friday, September 26** before 11:59 PM*

virtual office hours from 9 AM to 11 AM on September 26

Assignment #4 (Bounding Volume Hierarchy)

*due **Friday, October 10** before 11:59 PM*

virtual office hours from 4 PM to 6 PM on October 10

 *question? post a GitHub issue or email csc317tas@cs.toronto.edu*

As you saw in lecture, geometric queries like

```
answer = ...  
for object in objects:  
    // process object geometry  
    // update answer  
return answer
```

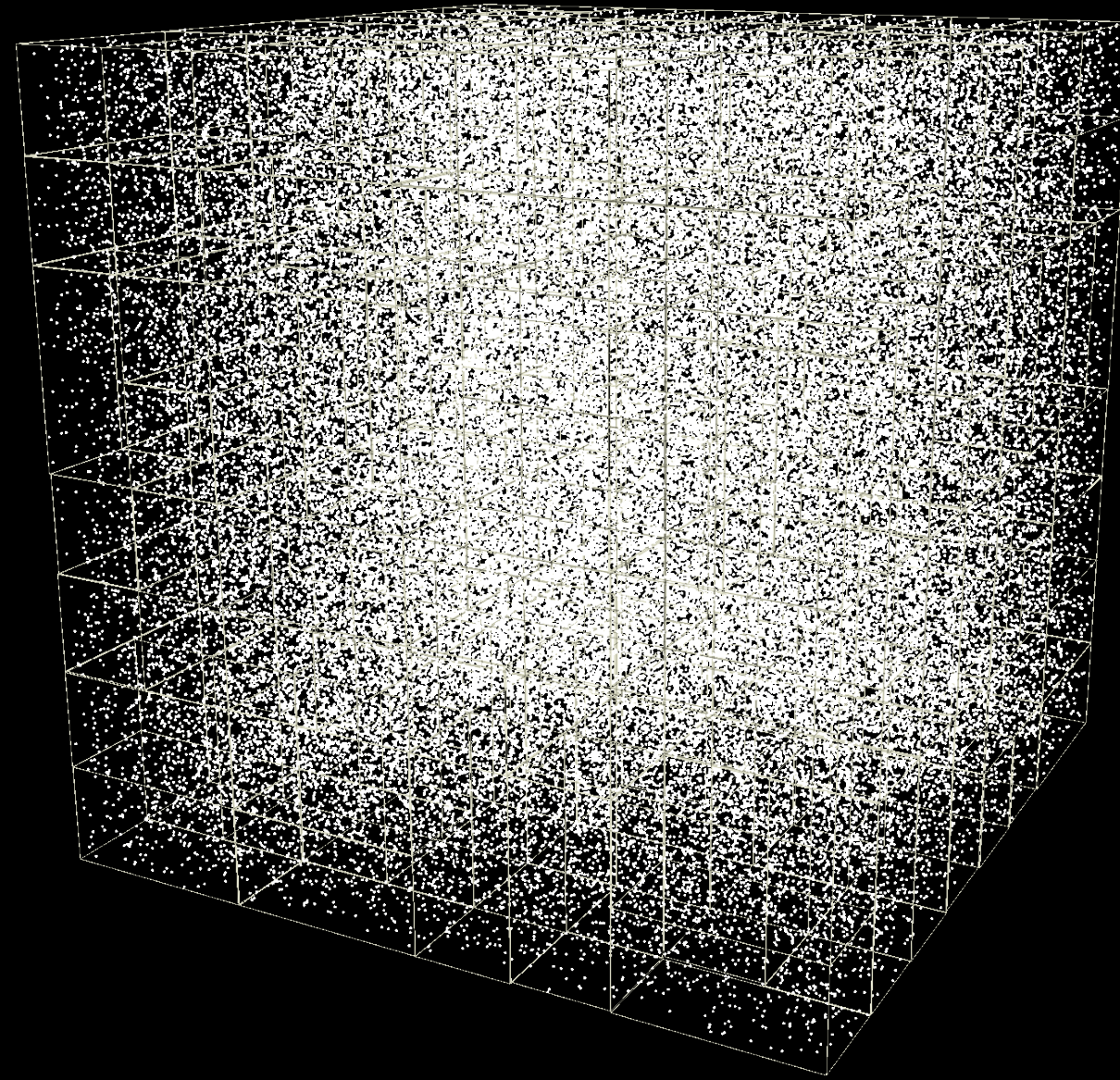
are slow!

In Assignment #4, you will speed up

ray intersection testing



closest point queries



collision testing



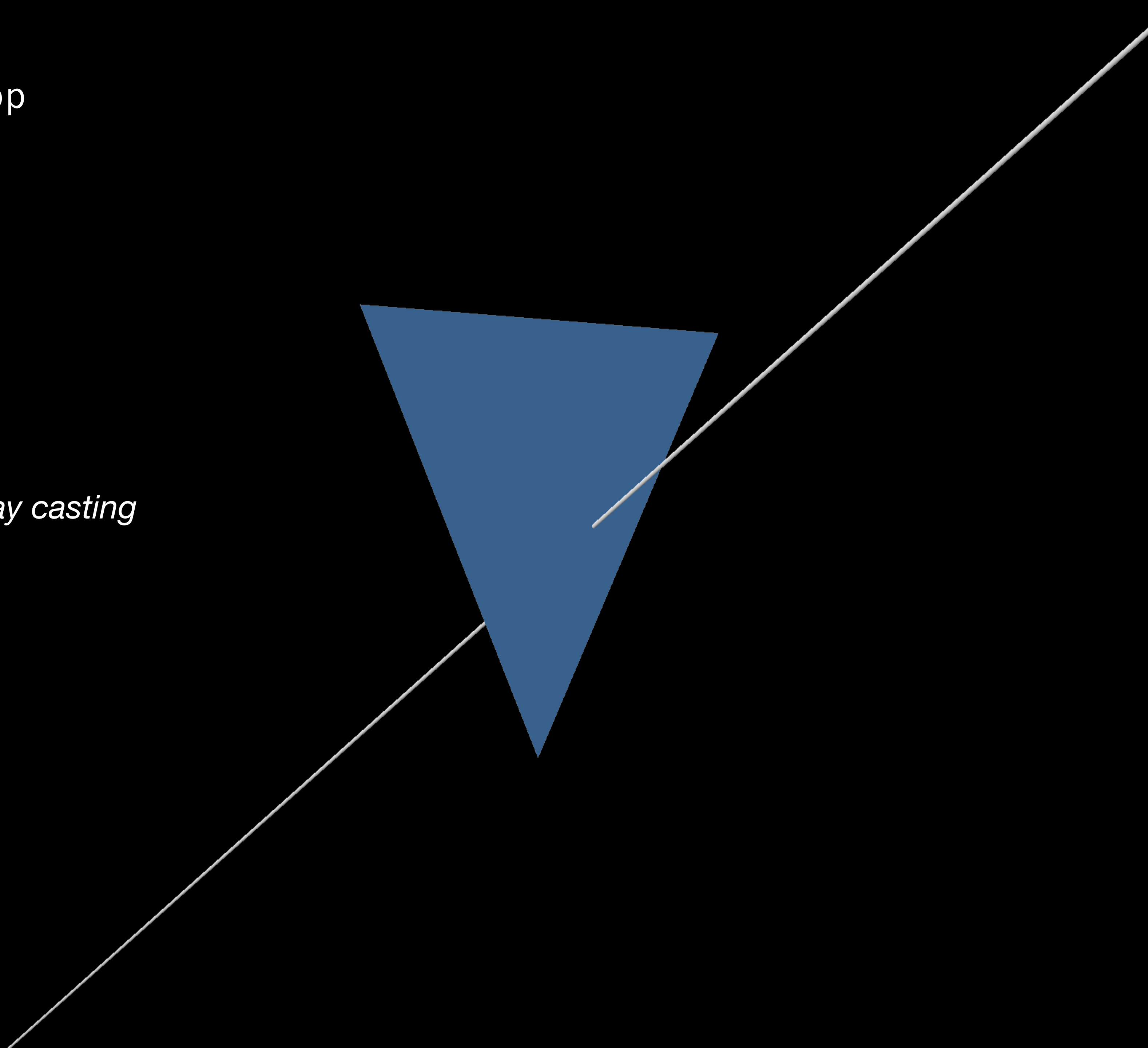
*using a **bounding volume hierarchy**.*

ray intersection testing



src/ray_intersect_triangle.cpp

Feel free to crib your solution from the ray casting assignment.




```
src/ray_intersect_triangle_mesh_brute_force.cpp
```

Aim for $O(n)$ complexity, focus on correctness.

You'll use this to validate your spatial acceleration structure!



What's an axis-aligned bounding box?

```
struct BoundingBox
{
    Eigen::RowVector3d min_corner;
    Eigen::RowVector3d max_corner;

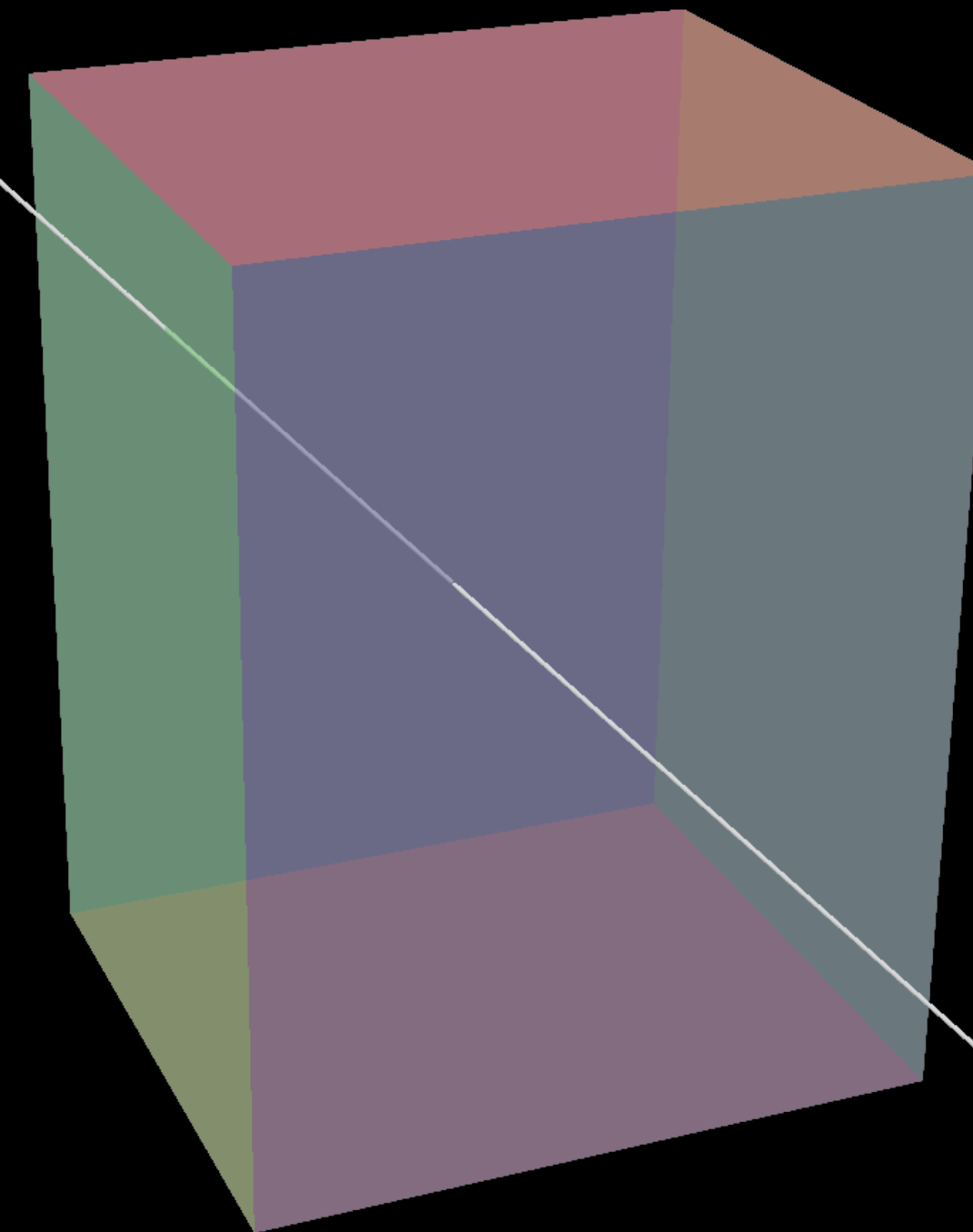
    Eigen::RowVector3d center()
    {
        return 0.5*(max_corner + min_corner);
    }
};
```

💡 *geometric queries (like ray intersection, point-box distances) can be reduced to queries on 1D intervals in the x, y, z dimensions.*

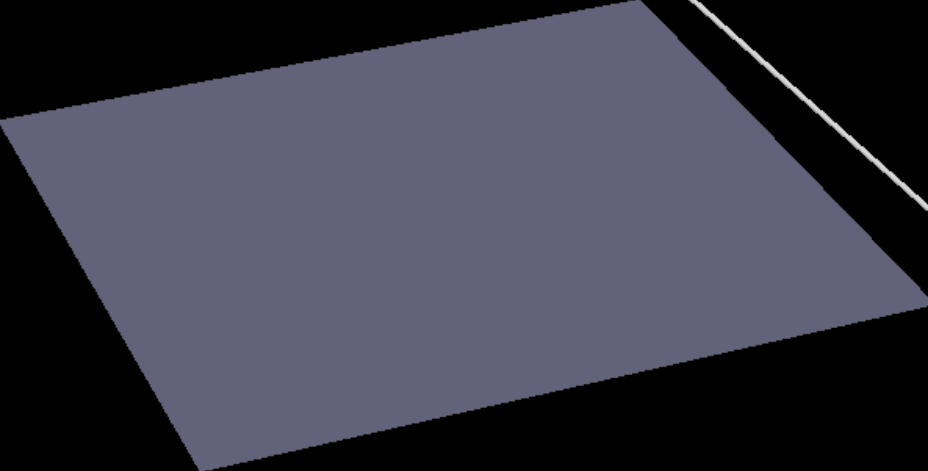
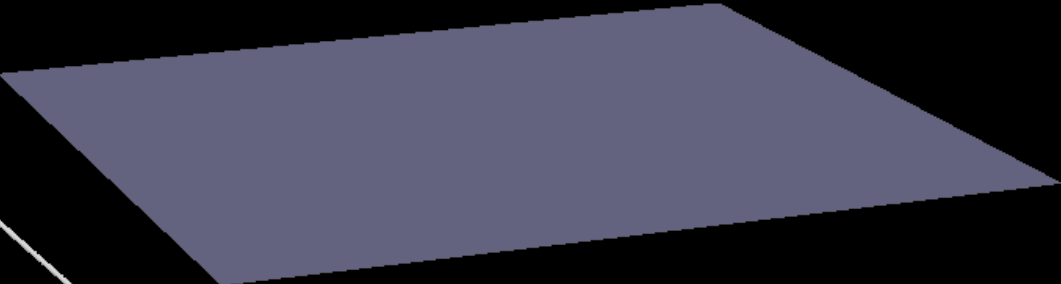
💡 *component-wise operations are your friends!*

src/ray_intersect_box.cpp

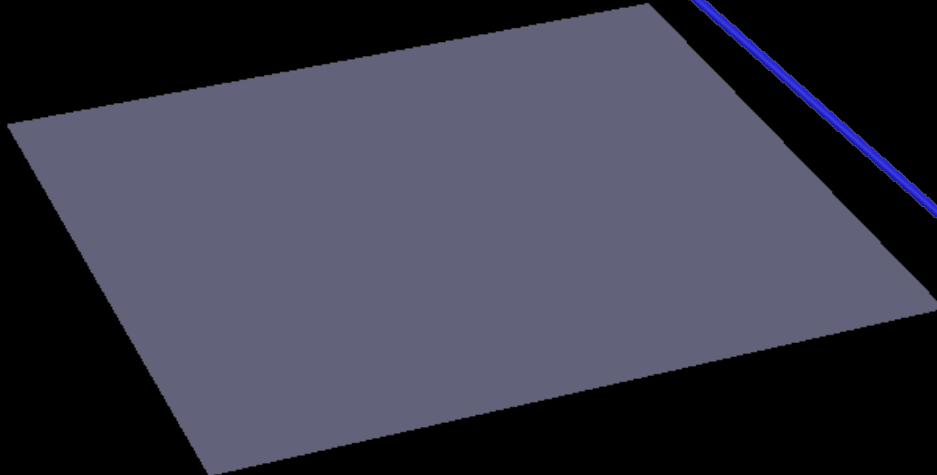
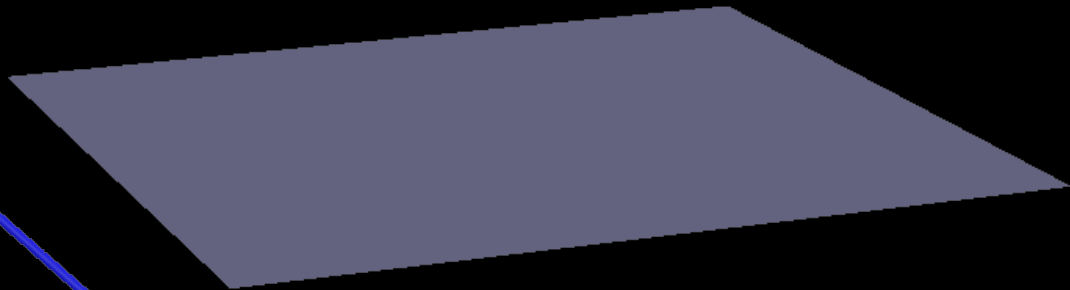
```
bool ray_intersect_box(  
    const Ray & ray,  
    const BoundingBox& box,  
    const double min_t,  
    const double max_t){  
  
    // return true if ray  
    intersects box at some  
    time min_t <= t <=  
    max_t.  
  
}
```



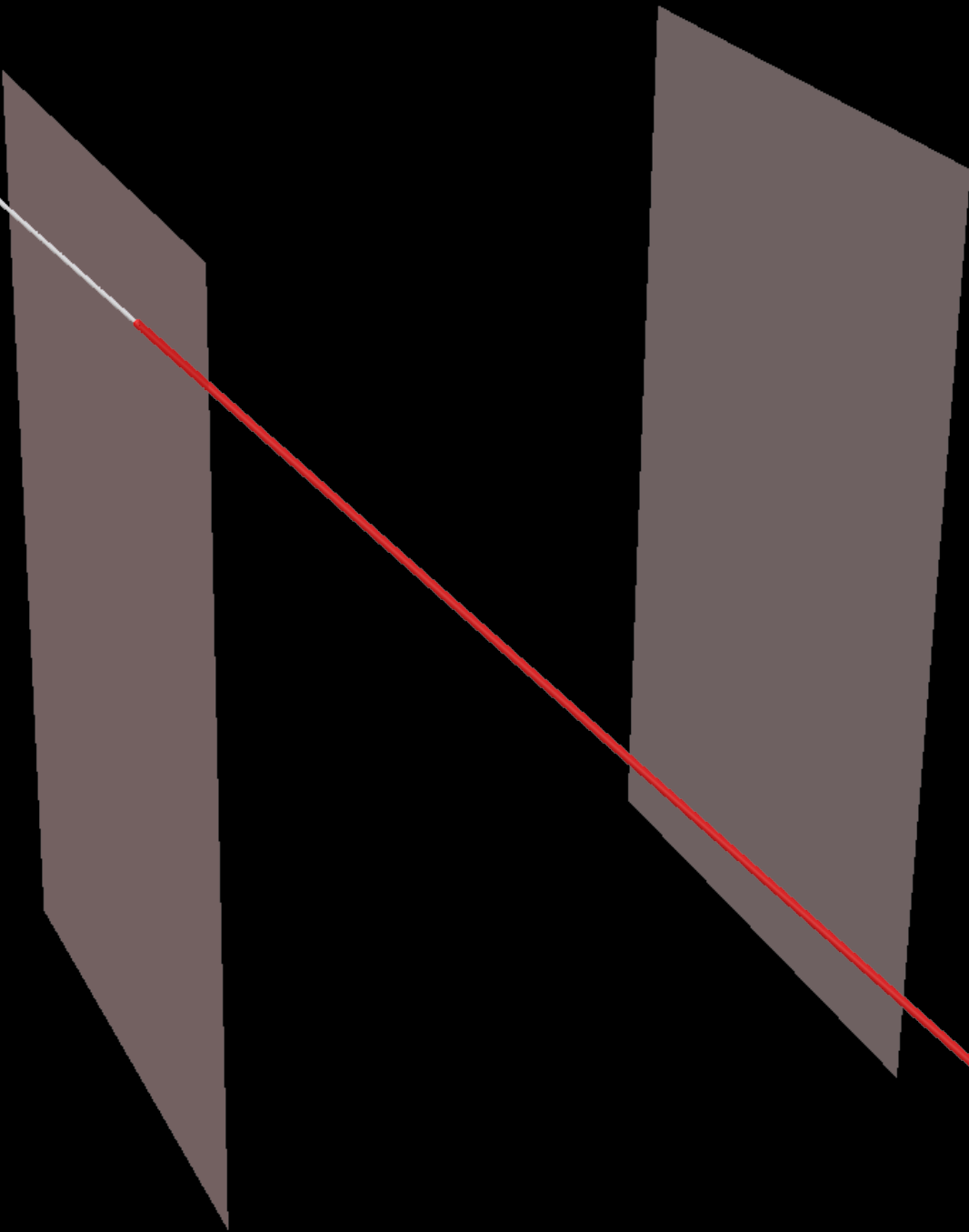
src/ray_intersect_box.cpp



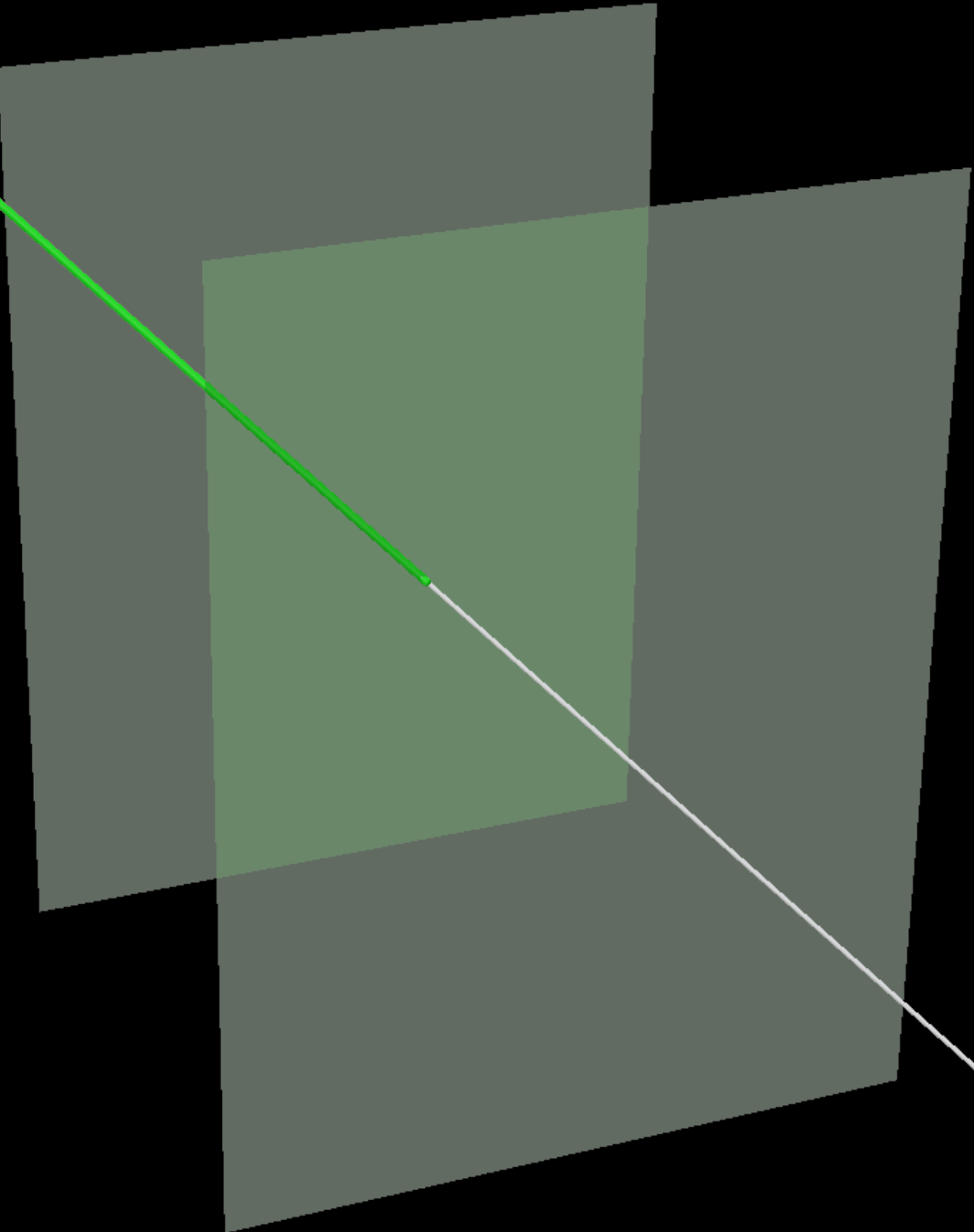
src/ray_intersect_box.cpp



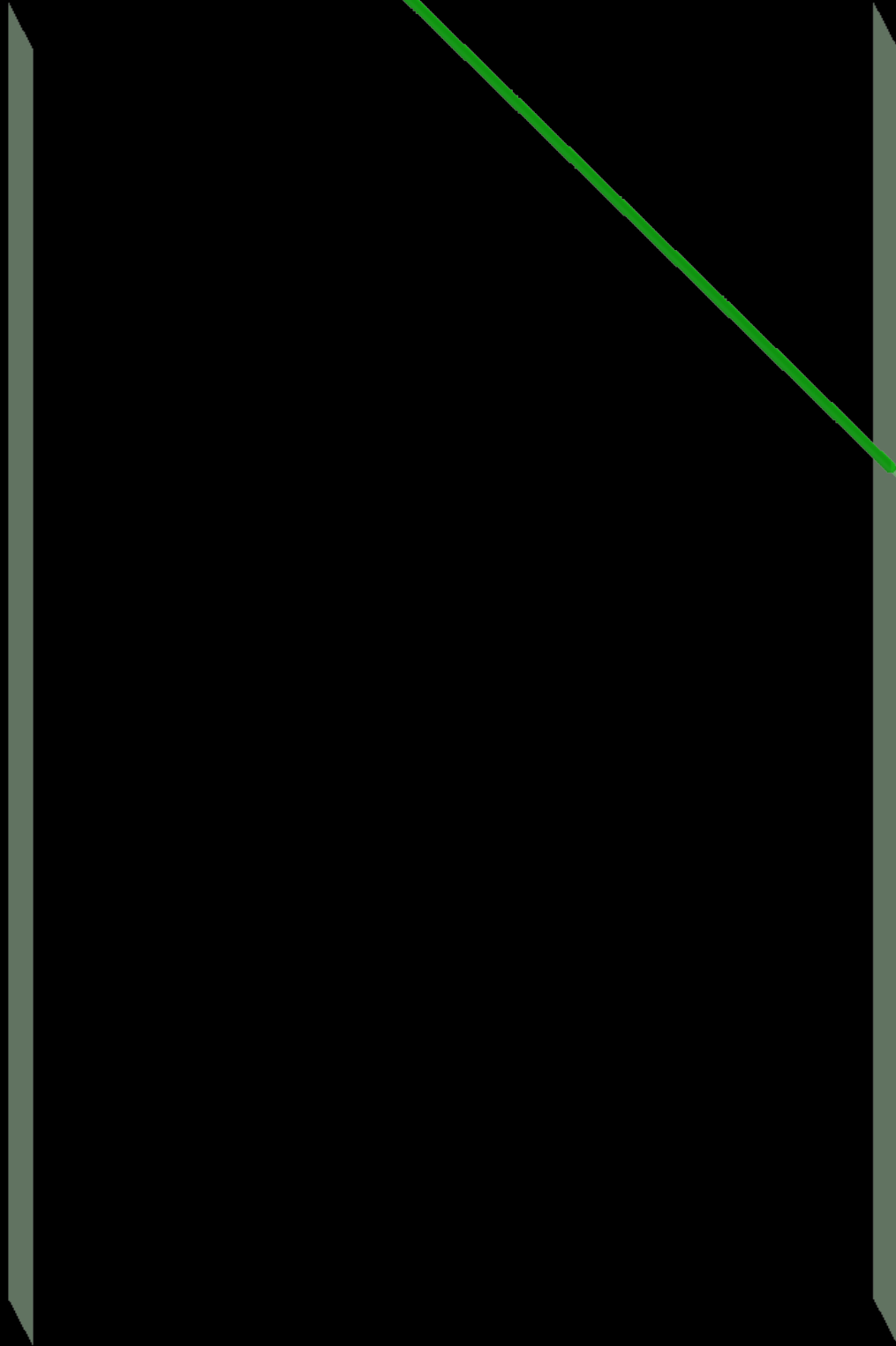
src/ray_intersect_box.cpp



src/ray_intersect_box.cpp



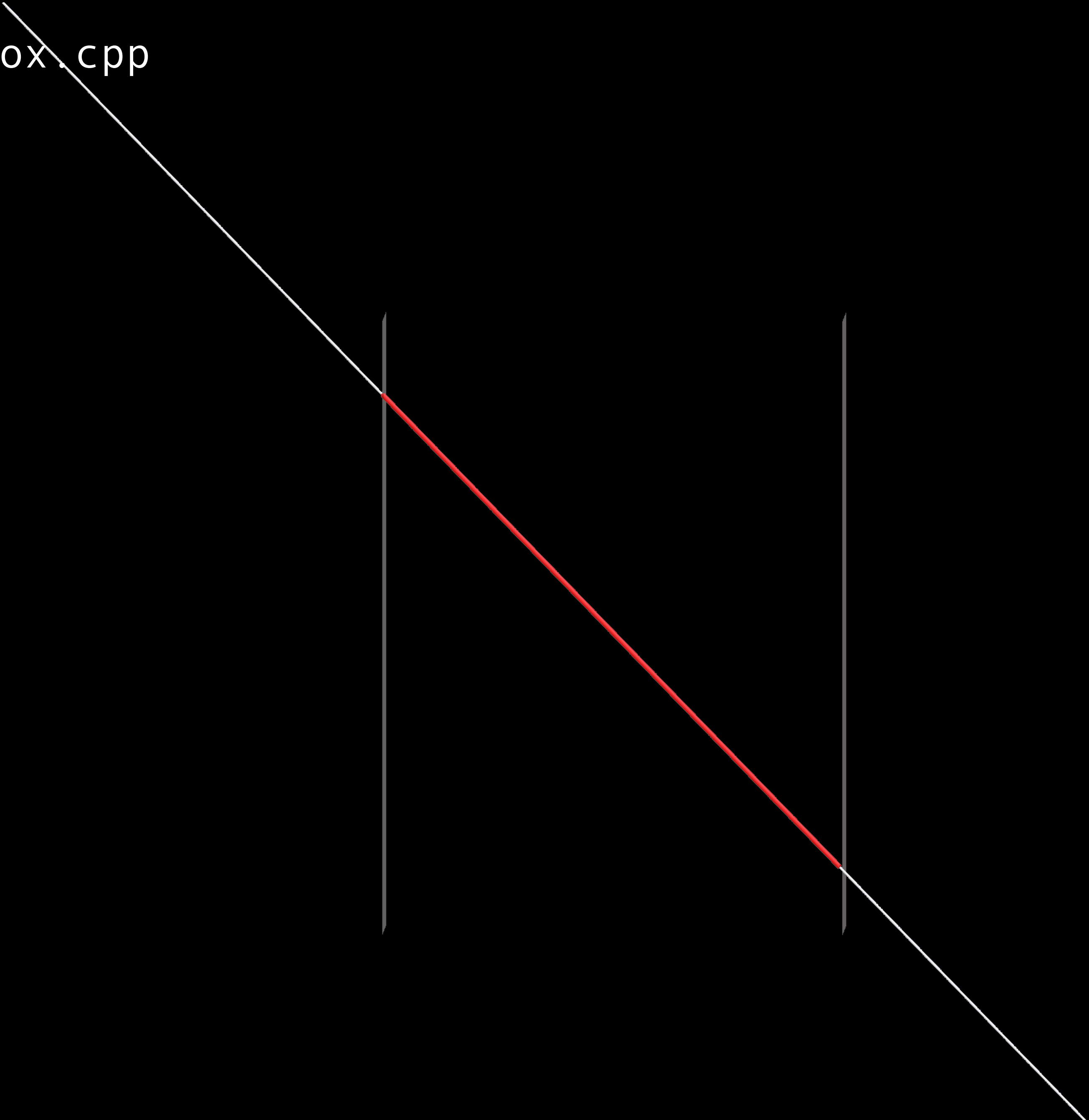
src/ray_intersect_box.cpp



src/ray_intersect_box.cpp



src/ray_intersect_box.cpp



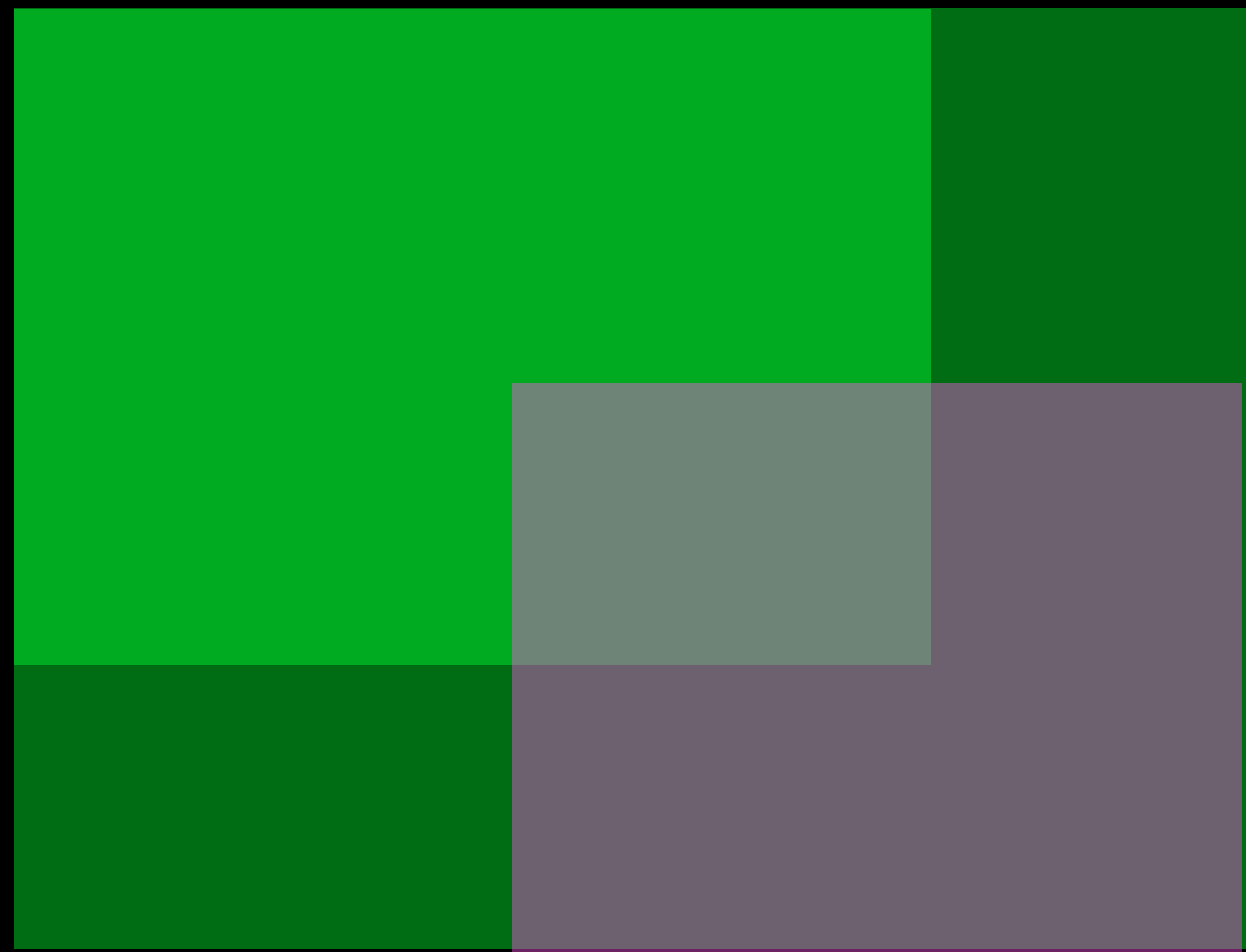
src/ray_intersect_box.cpp

```
bool ray_intersect_box(  
    const Ray & ray,  
    const BoundingBox& box,  
    const double min_t,  
    const double max_t){  
    // your code here  
}
```

Careful:

- *ray origin is in box -> hit* ✓
- *ray at min_t is in box -> hit* ✓
- *how might min_t and max_t enter this picture?*


```
void insert_box_into_box(
    const BoundingBox & A,
    BoundingBox & B)
{
}
```

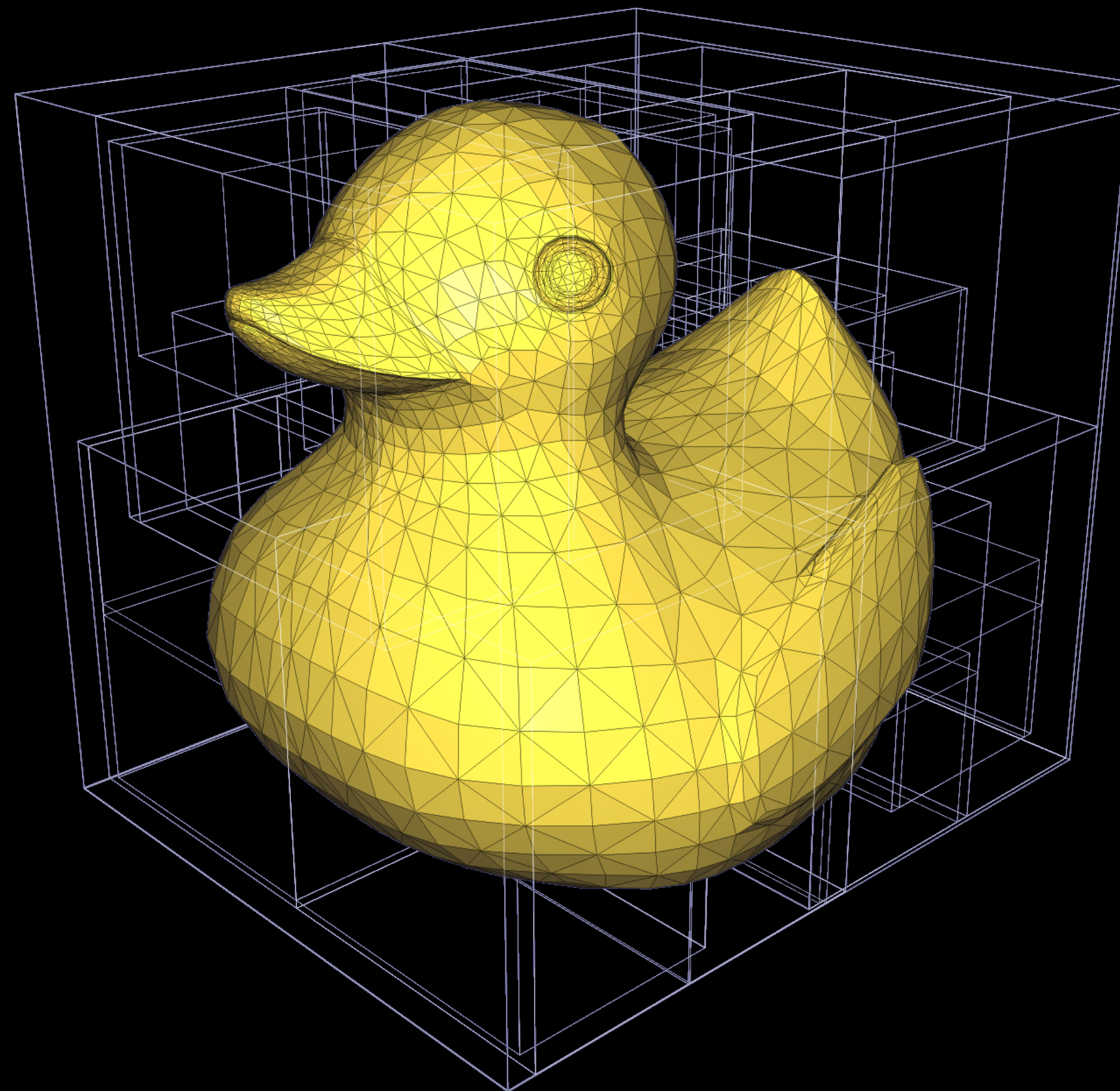


```
void insert_triangle_into_box(
    const Eigen::RowVector3d & a,
    const Eigen::RowVector3d & b,
    const Eigen::RowVector3d & c,
    BoundingBox & B)
{
}
```



✓ bounding volume

🤔 bounding volume hierarchy



AABBTree::AABBTree in src/AABBTree.cpp

```
struct AABBTree : public Object
{
    BoundingBox box; // inherited from Object

    std::shared_ptr<Object> left;
    std::shared_ptr<Object> right;

    int depth;
    int num_leaves;
};
```

your task is to write its constructor:

```
AABBTree(const std::vector<std::shared_ptr<Object> > & objects, int depth=0)
```



```
AABBTree(const
std::vector<std::shared_ptr<Object>> & objects,
int depth=0) {
```

👉 *splits the longest axis and divides objects equally.*

↻ *recursion?*

⊥ *base case? zero objects? one object?*

⚠ *a strange case you may encounter: all objects have the same coordinate across the longest axis...*

```
}
```

```
struct AABBTree : public Object
{
    BoundingBox box; // inherited from Object

    std::shared_ptr<Object> left;
    std::shared_ptr<Object> right;

    int depth;
    int num_leaves;
};
```


AABBTree::ray_intersect in src/AABBTree_ray_intersect.cpp

```
bool ray_intersect(
    const Ray& ray,
    const double min_t,
    const double max_t,
    double & t,
    std::shared_ptr<Object> & descendant) const
{
```

`std::dynamic_pointer_cast`

🌳 *how could I know if this->left is an AABBTree or Triangle or CloudPoint?*

👤 *when do I decide to recurse on my children?*

👤 *how does my child send me the intersected object?*

🔪 *how do I ensure I find the closest intersected object?*

```
struct AABBTree : public Object
{
    BoundingBox box; // inherited from Object

    std::shared_ptr<Object> left;
    std::shared_ptr<Object> right;

    int depth;
    int num_leaves;
};
```

```
}
```

build_release — -zsh — 84x12

...c317/computer-graphics-bounding-volume-hierarchy/build_release — -zsh

(base) nitvishn@macvishn-250 build_release % ./rays

[# Ray Triangle Mesh Intersection]

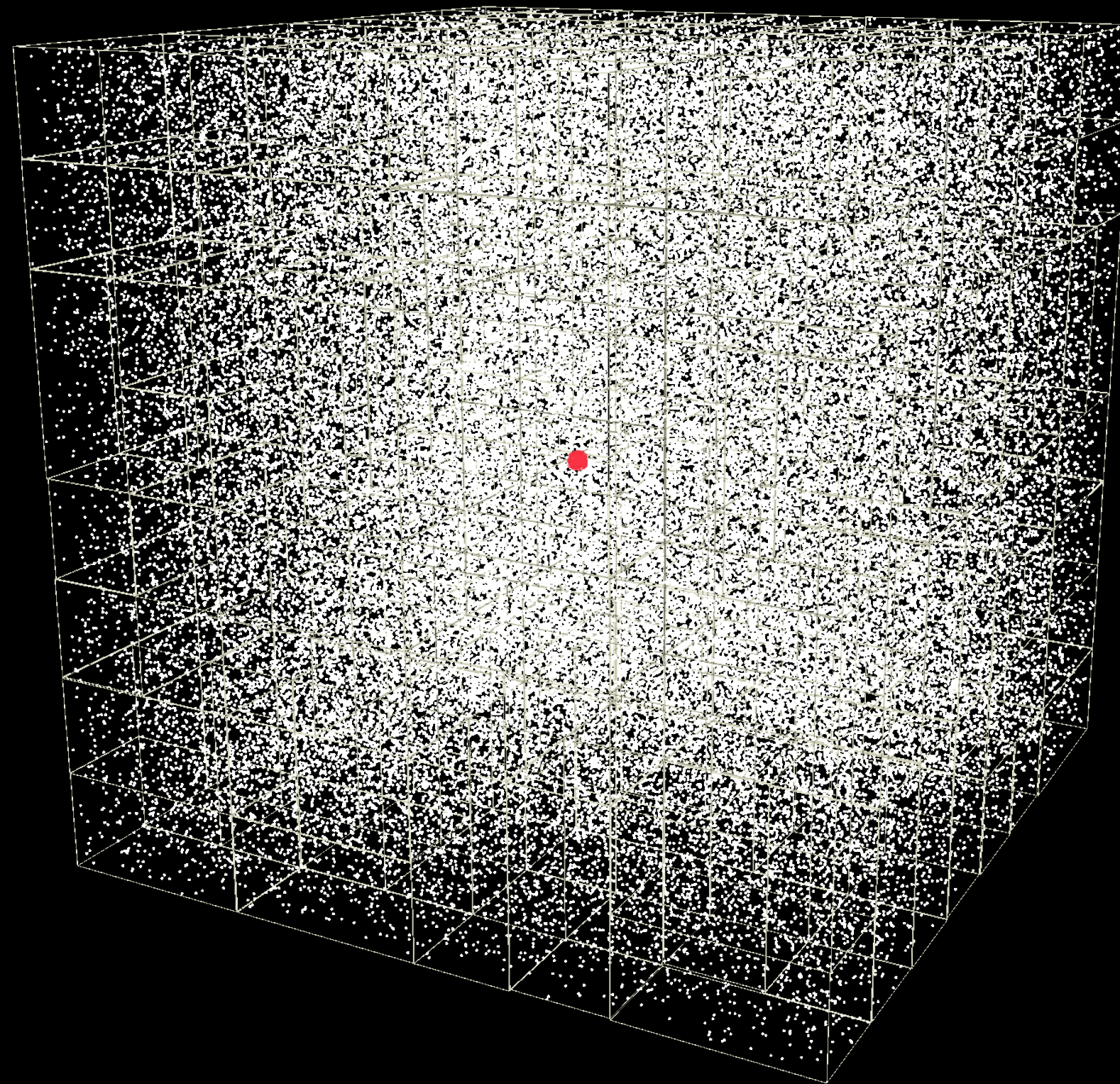
|V| 2050

|F| 4096

Firing 1000 rays...

Method	Time in seconds
brute force	0.03428411484
build tree	0.00162792206
use tree	0.00434708595

closest point queries




```
src/nearest_neighbor_brute_force.cpp
```

```
void nearest_neighbor_brute_force(
    const Eigen::MatrixXd & points,
    const Eigen::RowVector3d & query,
    int & I,
    double & sqrD)
{

}
```


src/point_box_squared_distance.cpp

```
double  
point_box_squared_distance(  
    const Eigen::RowVector3d &  
query,  
    const BoundingBox & box)  
{
```

 *exploit axis-alignedness!*

```
}
```

src/point_AABBTree_squared_distance.cpp

```
// when this returns, descendant is the closest leaf object to query.  
//     descendant is at squared distance sqrd.  
//     also, min_sqrd <= sqrd <= max_sqrd.
```

```
bool point_AABBTree_squared_distance(  
    const Eigen::RowVector3d & query,  
    const std::shared_ptr<AABBTree> & root,  
    const double min_sqrd,  
    const double max_sqrd,  
    double & sqrd,  
    std::shared_ptr<Object> & descendant)  
{
```

```
struct AABBTree : public Object  
{  
    BoundingBox box; // inherited from Object  
  
    std::shared_ptr<Object> left;  
    std::shared_ptr<Object> right;  
  
    int depth;  
    int num_leaves;  
};
```

Acceleration principle: we would like to eliminate entire subtrees by declaring that

any point in this subtree is farther than the best point seen so far.

```
}
```

```
// when this returns, descendant is the closest leaf object to query.
//     descendant is at squared distance sqrd.
//     also, min_sqrd <= sqrd <= max_sqrd.

bool point_AABBTree_squared_distance(query, root, min_sqrd, max_sqrd, sqrd, descendant)
{
```

Acceleration principle: we would like to eliminate entire subtrees by declaring that

any point in this subtree is farther than the best point seen so far.

- *what could you check that would guarantee this?*
- *in what order should we process nodes of the AABBTree?*

breadth-first?

depth-first?

somethingelse-first?

```
}
```

```
struct AABBTree : public Object
{
    BoundingBox box; // inherited from Object

    std::shared_ptr<Object> left;
    std::shared_ptr<Object> right;

    int depth;
    int num_leaves;
};
```



```
// when this returns, descendant is the closest leaf object to query.  
//     descendant is at squared distance sqrd.  
//     also, min_sqrd <= sqrd <= max_sqrd.
```

```
bool point_AABBTree_squared_distance(query, root, min_sqrd, max_sqrd, sqrd, descendant)  
{
```

```
    // initialize a queue prioritized by minimum distance  
    d_r ← distance to root's box  
    Q.insert(d_r, root)
```

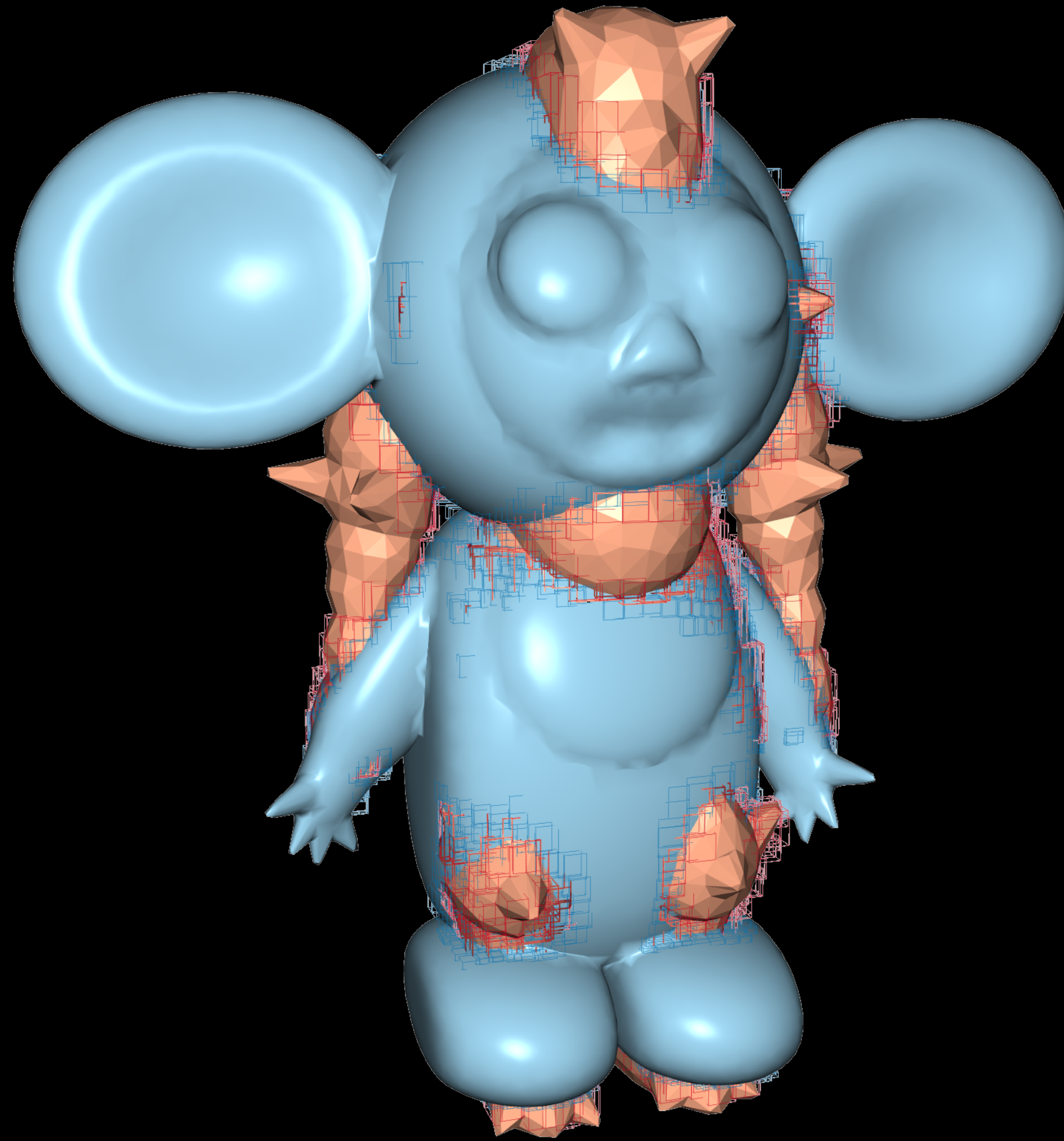
```
    // initialize minimum distance seen so far  
    d ← ∞
```

```
    while Q not empty  
        // d_s: distance from query to subtree's bounding box  
        (d_s, subtree) ← Q.pop  
        if d_s < d  
            if subtree is a leaf  
                d ← min[ d , distance from query to leaf object ]  
            else  
                d_l ← distance to left's box  
                Q.insert(d_l ,subtree.left)  
                d_r ← distance to right's box  
                Q.insert(d_r ,subtree.right)
```

```
}
```

```
struct AABBTree : public Object  
{  
    BoundingBox box; // inherited from Object  
  
    std::shared_ptr<Object> left;  
    std::shared_ptr<Object> right;  
  
    int depth;  
    int num_leaves;  
};
```


collision testing



src/triangle_triangle_intersection.cpp

T_1 intersects $T_2 \iff$ at least one tri edge intersects the other tri.

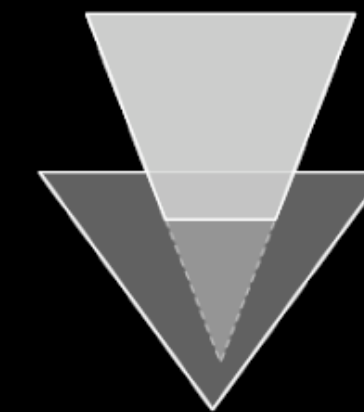
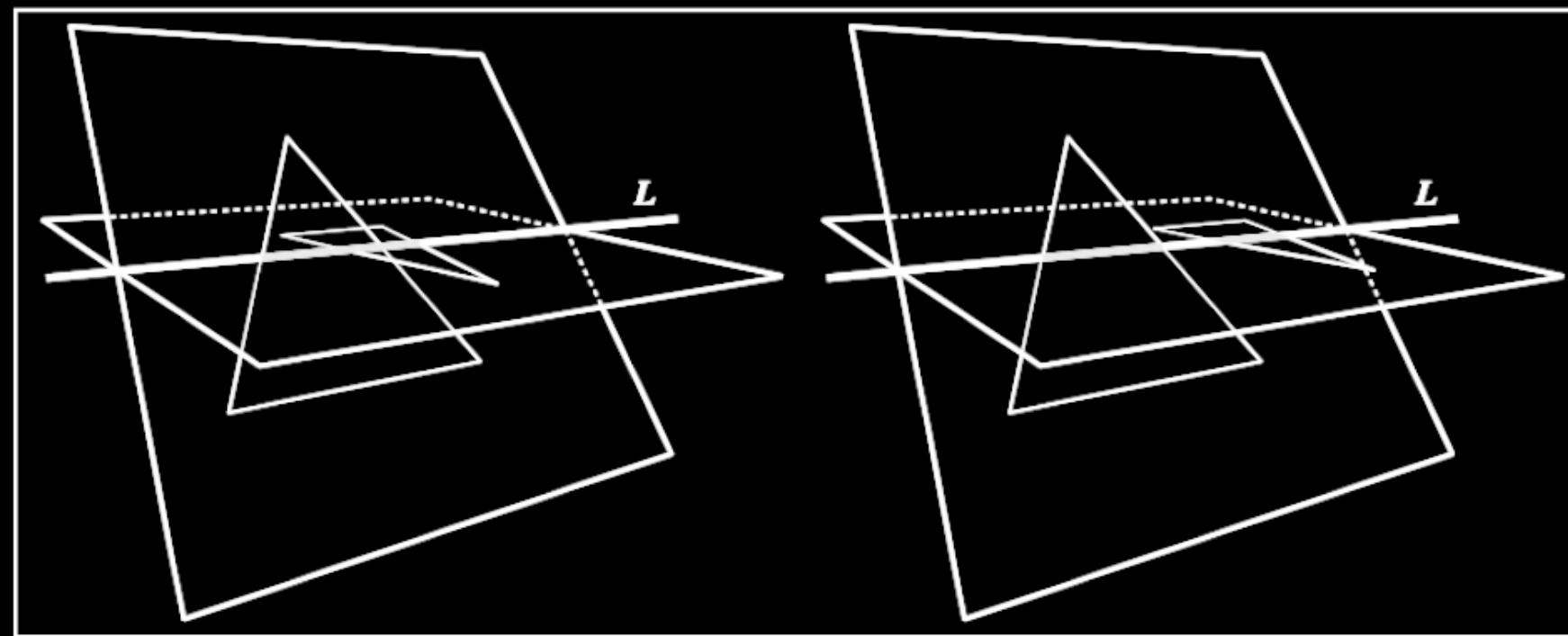
Algorithm 1: Test edge-tri intersection for all 6 edges.

T_1 intersects $T_2 \Rightarrow$ Vertices of T_1, T_2 straddle plane of T_2, T_1 respectively.

What if T_1, T_2 are co-planar?

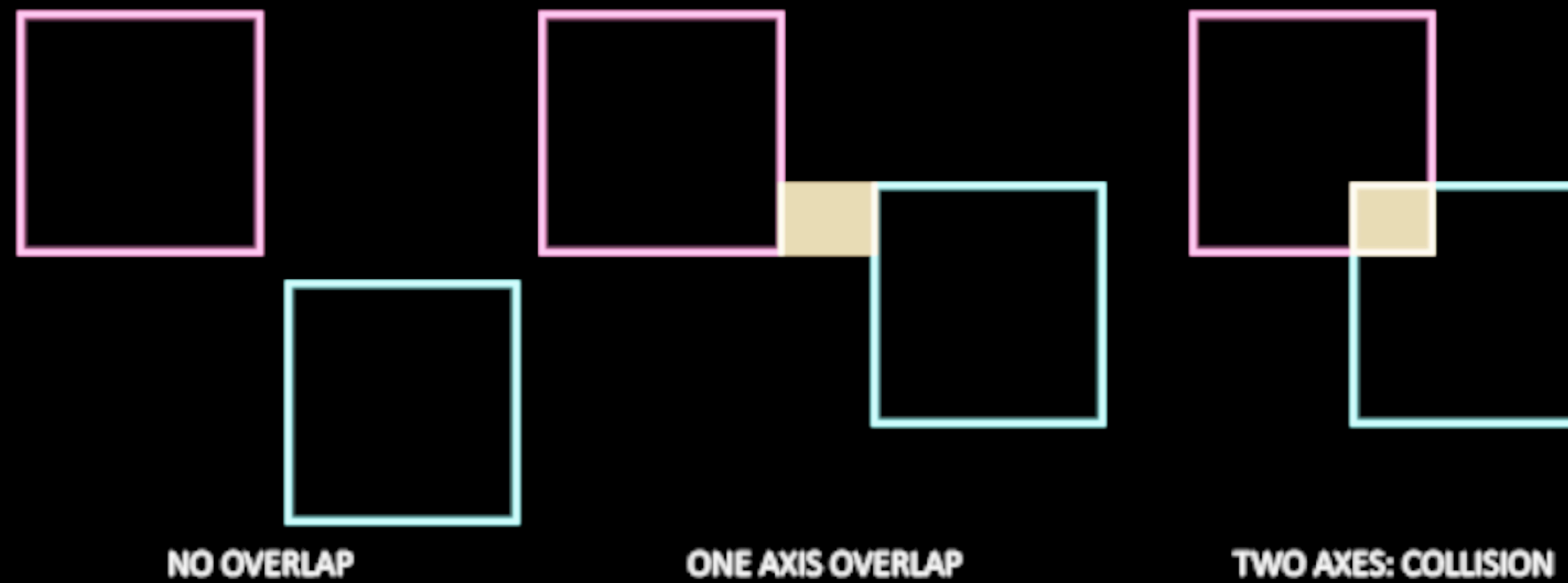
Algorithm 2: if (vertices of T_1, T_2 on the same side of plane of T_2, T_1) return false;

The triangles intersect iff the triangle intervals along line of intersection do.



src/box_box_intersect.cpp

as always with axis-aligned bounding boxes, the problem splits component-wise.




```
void find_all_intersecting_pairs_using_AABBTrees(  
    const std::shared_ptr<AABBTree> & rootA,  
    const std::shared_ptr<AABBTree> & rootB,  
    std::vector<std::pair<std::shared_ptr<Object>, std::shared_ptr<Object> > > &  
        leaf_pairs)  
{
```

*Acceleration principle: we would like to eliminate **pairs** of subtrees (A, B) by declaring that*

any object in this subtree A does not intersect any object in subtree B.

```
}
```



```
void find_all_intersecting_pairs_using_AABBTrees(  
    const std::shared_ptr<AABBTree> & rootA,  
    const std::shared_ptr<AABBTree> & rootB,  
    std::vector<std::pair<std::shared_ptr<Object>, std::shared_ptr<Object> > > &  
        leaf_pairs)  
{
```

```
    leaf_pairs <- {}
```

```
    Q <- empty queue of pairs to process
```

```
    add (rootA, rootB) to Q if their boxes intersect
```

```
    while Q not empty:
```

```
        pop (A, B) from Q
```

```
        if they're both leaves:
```

```
            add (A, B) to leaf_pairs
```

```
        else:
```

```
            add new relevant children pairs to Q
```

```
}
```

Good luck!